

GEDCOM Parser System Design and Enrichment

This system will parse, normalize, and enrich GEDCOM files in a structured pipeline. Core tasks include **decomposing NAME fields** into GEDCOM sub-tags (e.g. NPFX, GIVN, SURN, NSFX, etc.), standardizing places with geocoordinates, extracting occupations, and linking to external data. All fields should be fully populated (even creating missing child tags) to achieve a *canonical* GEDCOM format. Below is a comprehensive design outline:

1. Name Normalization and Tagging

- **GEDCOM Name Structure:** Per GEDCOM 5.5.1, the NAME line is split into sub-fields in this order: NPFX (prefix), GIVN (given name), NICK (nickname), SPFX (surname prefix), SURN (surname), NSFX (suffix) ¹. For example, `NAME William D. "Bill" /van der Berg/ Jr.` yields `GIVN="William D.", NICK="Bill", SPFX="van der", SURN="Berg", NSFX="Jr."` ¹. The parser should follow this standard order when splitting a full NAME string, ensuring every record has these tags even if originally absent. This handles cases where one genealogy tool exports only `1 NAME` and another exports all sub-tags; our parser should populate both.
- **Tokenization & Parsing:** Tokenize the raw NAME text (handling quotes, slashes, commas). Use heuristic rules and dictionaries (name prefixes, suffix lists, titles, nicknames) to classify tokens. For example, in `SGT Mr. Norman Forsyth Menzies Jr.`: `"SGT"` can map to a rank title (possibly mapped to TITL or NPFX), `"Mr."` is a name prefix (NPFX), `"Norman Forsyth"` are given/middle names (GIVN), `"Menzies"` is the surname (SURN), `"Jr"` is suffix (NSFX). The system uses JSON datasets (e.g. `name_prefixes.json`, `name_variants.json`, `nicknames.json`, `titles_normalization.json`) to recognize and normalize these parts. Configuration (e.g. `processing_config.yml`) enables name normalization and loads these datasets ².
- **Custom Titles and Ranks:** Military ranks or other titles (like "SGT") can be treated as GEDCOM TITL or as additional prefix. A normalization database of ranks (e.g. `military_rank_standardization.json`) and titles (e.g. `titles_normalization.json`) helps map custom tokens to standardized forms. The parser should optionally tag these either as TITL or as NPFX according to conventions (for instance, using TITL for rank and NPFX for civilian titles).
- **Nicknames and Phonetics:** The system can generate nicknames and phonetic variants for enrichment. A YAML config (`enrich_names_config.yml`) lists enabled romanization schemes (e.g. Pinyin, Hepburn) and phonetic algorithms (Soundex, Metaphone, etc) ³. This allows optional creation of alternate name forms for matching.

2. Place Standardization and Geolocation

- **Hierarchy Parsing:** GEDCOM `PLAC` fields often use comma-separated locality hierarchy (city, county, state, country). Parse each part and store them consistently (e.g. use separate columns in the database for city, state, country, or store the full string but indexed). Use postal/geo databases to standardize names (e.g. expand "N.Y." to "New York"). A dataset of state/country names (e.g. `us_states.json`) aids normalization.

- **Geocoding:** For every distinct place, use a geocoding service (e.g. GeoNames or OpenStreetMap) to obtain latitude/longitude. Store these as MAP coordinates under `LATI` / `LONG` tags or in the database. This enables mapping and spatial queries. The database layer includes tables for **distinct places** with latitude/longitude and for **place occurrences** (linking events to places) to avoid duplication.
- **Place Registry:** Build a canonical place registry (in PostgreSQL) that clusters slightly different place strings into single entries. Use fuzzy matching or Geonames queries to merge variants (e.g. “NY, USA” and “New York, United States”). The project’s SQL scripts (like `002_geonames_candidates.sql` and `004_distinct_places.sql`) implement this logic. Store place hierarchies (country/state/county) to support both relational (Postgres) and graph (Neo4j) queries.
- **GIS/Mapping:** With lat/long available, feed place data to Neo4j (for graph queries) and to any web map widgets. This enriches visualizations (e.g. mapping an ancestor’s life events).

3. Occupation and Other Tag Enrichment

- **OCCU Tag Population:** If an `OCCU` tag is already present, normalize it (e.g. “farmer” vs “Farmer”). If not present but the word “Occupation:” appears in an INDIVIDUAL’s NOTE, extract the following text as the occupation. Use keyword lists (`occupation_keywords.json`, `occupations.json`) to identify and standardize the extracted term.
- **Additional Fields:** Similarly, parse other custom fields (e.g. RELI for religion, TITL for titles) using corresponding datasets (e.g. `religious.json`, `prefix_draft.json`, `royal.json`). The pipeline should optionally enable modules for historical military data, royalty titles, and religious affiliations ⁴. When enabled, each module enriches relevant tags (e.g. mapping a noble rank in NAME to a standard title, or flagging known military units in events).

4. Pipeline Configuration and Execution

- **Config-Driven Pipeline:** A YAML config controls which steps run. For example, `processing_config.yml` can turn on **name normalization**, **date normalization**, **place normalization**, etc. It also lists which datasets to load (e.g. `name_normalization`, `nicknames`, `titles`) and which enrichment modules to use (military, royalty, religion) ² ⁵. This makes every feature optional/configurable.
- **CLI Interface:** The core engine is a command-line tool (built with Typer). Commands include `parse`, `normalize`, `export`, `doctor` (for checking), etc. An example CLI stub is shown in `app.py`. The CLI loads the GEDCOM, builds an in-memory registry, runs enabled pipeline steps, and can export to JSON or load into DB. Users (genealogists) can script or use the CLI to process files in batch.
- **Integration:** After parsing, the data flows into PostgreSQL and Neo4j. Postgres holds normalized entities (persons, families, events, places) in tables. Neo4j holds a graph of individuals and relationships for complex queries. The pipeline ensures referential integrity (e.g. unique person IDs).

5. External Data and APIs

- **Datasets:** The system leverages curated JSON datasets (as seen under `datasets/`) for normalization (names, occupations, places). These are version-controlled and can be expanded (e.g.

adding a new country's military ranks). The `files:` section of the config maps dataset names to files ⁶.

- **Wikidata and Online Sources:** For further enrichment, the system can query external APIs. For example, normalize place names via Wikidata or GeoNames to match known entities. Person names could link to Wikidata items to fetch life dates or alternate spellings. If an ancestor's unique name is found on Wikipedia/Wikidata, one could pull their birth date or biography. The system can expose REST API endpoints (or use existing APIs) to allow front-end UI or other tools to query the normalized database.
- **Exporting:** Besides database storage, a JSON export (via a CLI command) outputs all normalized data for other uses (e.g. importing into different genealogy software). The export command in `export.py` shows how individuals, families, notes, etc., are serialized. Additional exports (e.g. GEDCOM generation) can be added.

6. User Interface & Tools

- **CLI First, GUI Later:** The primary interface is a CLI, but a web UI is planned. The web interface (server-hosted) will let genealogists upload GEDCOMs, configure normalization options via forms (reflecting the YAML config), and view results (trees, maps, charts). It can also support interactive queries (e.g. "find all individuals with surname X and occupation Y").
- **Documentation and Templates:** Use YAML and Markdown for configuration and documentation. Examples and templates (shown in the repository's `docs/` folder) guide users on config options and pipeline design. Inline code documentation and a user manual will cover best practices (e.g. GEDCOM conventions, how child tags map to NAME). Open standards (GEDCOM 5.5.1, SQL, Cypher) are documented for future developers.
- **Testing & Validation:** Include a `--validate` mode to check GEDCOM conformance and catch missing data. Unit tests (some exist under `tests/`) cover parsing edge cases (UTF-8, unusual tags). Continuous integration should run these tests on each change.

7. Standards and Best Practices

- **GEDCOM Compliance:** Target GEDCOM 5.5.1 as the canonical format. Follow its tag hierarchy and order ¹. For unknown custom tags, either map them into standard tags or store them in a generic TAG/NOTE with metadata.
- **Database Design:** Normalize the schema in Postgres – e.g. separate tables for individuals, families, events, places – linking via foreign keys. Use indexes on key fields (names, dates, place IDs) to speed queries. Neo4j nodes should be labeled (Person, Family, Place) and relationships (PARENT_OF, MARRIED_TO, LOCATED_AT). Naming conventions (snake_case for SQL, CamelCase for Neo4j) should be documented.
- **Modularity:** Organize the codebase into modules (we see directories for `normalization`, `enrichment`, `entities`, etc.). Each normalization type (name, date, place) should be a separate component. This allows future extension (e.g. DNA data integration) without monolithic code.
- **Security and Privacy:** If deployed, ensure GEDCOM uploads and data storage are secure (authentication, encryption at rest). Provide options to strip sensitive data (e.g. living individuals) during processing.

This design leverages the existing code infrastructure (datasets, SQL scripts, config files) and follows known genealogy data standards. It will **fully enrich GEDCOMs** by populating all sub-tags (even generating new ones) and linking every field to standardized references, as configured by the user ¹ ² .

Sources: GEDCOM spec and parser docs ¹ ; project configuration examples ² ⁵ . (All cited lines are from the repository and related documentation.)

¹ Import from another genealogy program - Gramps

https://www.gramps-project.org/wiki/index.php/Import_from_another_genealogy_program

² ⁴ ⁵ ⁶ processing_config.yml

file:///file_000000006a2c7230bcd23edd4675d3d1

³ enrich_names_config.yml

file:///file_000000004124723091296951d3968243